

DIPLOMA DATA ENGINEER PYTHON FOR DATA ENGINEERING

Docente: Angel Tintaya



REGLAS



Se requiere **puntualidad** para un mejor desarrollo del curso.



Para una mayor concentración **mantener silenciado el micrófono** durante la sesión.



Las preguntas se realizarán **a través del chat** y en caso de que lo requieran **podrán activar el micrófono**.



Realizar las actividades y/o tareas encomendadas en **los plazos determinados**.



Identificarse en la sala Zoom con el primer nombre y primer apellido.



EVALUACIÓN

Asistencia (Curso):

mínimo 80% sesiones para
recibir la certificación

Participación
(20%) + ***Caso 1***
(10%) + ***Caso 2***
(10%) + ***Exámen Final***
(60%)



OBJETIVO

General:

- Aprender a utilizar el lenguaje Python para el desarrollo de procesos ETL.

Específicos:

- Dominar Python como lenguaje de programación.
- Entender y poner en práctica el desarrollo de procesos ETL utilizando Python.



Pandas - I



Pandas: Series y Dataframe

- ¿Qué es Pandas?
- Estructura de datos con Pandas
- Pandas y Numpy
- Creación y guardado de un Dataframe
- Analizar datos con Pandas
- Propiedades de un Dataframe
- Trabajando con Series
- Seleccionando campos

Pandas: Series y Dataframe

¿Qué es Pandas?

- Es una librería de código abierto que ofrece unas estructuras muy poderosas y flexibles que facilitan la manipulación y tratamiento de datos.



Pandas: Series y Dataframe

Estructura de datos con Pandas

- Las principales estructura de datos de pandas son: Series y Dataframe

Series			Series			DataFrame	
	apples			oranges		apples	oranges
0	3	+	0	0	=	0	0
1	2		1	3		1	3
2	0		2	7		2	7
3	1		3	2		3	2



Pandas: Series y Dataframe

Series:

- Arreglo unidimensional etiquetado capaz de almacenar cualquier tipo de dato.

```
Type: <class 'pandas.core.series.Series'>
0    Bulbasaur
1     Ivysaur
2    Venusaur
Name: Name, dtype: object
```

Dataframe:

- Estructura bidimensional con columnas que pueden ser también de cualquier tipo. Estas columnas son a su vez Series.

```
Type: <class 'pandas.core.frame.DataFrame'>
```

	Name	Type
0	Bulbasaur	Grass
1	Ivysaur	Grass
2	Venusaur	Grass

Pandas: Series y Dataframe

Pandas y Numpy:

- Numpy es una librería especializada en el cálculo numérico y el análisis de datos, especialmente para un gran volumen de datos.
- Utilizaremos esta librería junto a pandas principalmente para definir valores faltantes (NaN - Not a Number): `np.nan`



Pandas: Series y Dataframe

- Creación de un Dataframe: Desde una ruta (/content/drive/MyDrive/Datasets/pokemon.txt)

```
df_drive_pkmn = pd.read_csv('/content/drive/MyDrive/Datasets/pokemon.txt')
print(type(df_drive_pkmn))
df_drive_pkmn.head(2)
```

```
<class 'pandas.core.frame.DataFrame'>
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False



Pandas: Series y Dataframe

- Creación de un Dataframe: Desde un link
(<https://raw.githubusercontent.com/AngelTintaya/datasets/main/pokemon.csv>)

```
df_link_pokemon = pd.read_csv('https://raw.githubusercontent.com/AngelTintaya/datasets/main/pokemon.csv')
print(type(df_link_pokemon))
df_link_pokemon.head(2)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False



Pandas: Series y Dataframe

- Creación de un Dataframe: Desde un diccionario

```
dic_clase = {'codigo': [9876654, 7665498, 6598764, 5498766, 8976654],
            'edad': [26, np.nan, 30, 42, 38],
            'ingreso': [2500, 2500, 3500, 4800, 4100],
            'estado_civil': ['Soltero', 'Soltero', 'Casado', 'Casado', 'Soltero'],
            'fecmatricula': ['25/07/2022', '21/07/2022', '28/07/2022', '26/07/2022', '22/07/2022']}

print(type(dic_clase))

<class 'dict'>

df_clase = pd.DataFrame(dic_clase)
df_clase.head(2)
```



Pandas: Series y Dataframe

Analizar datos con Pandas:

Dataframe df_clase:

df_clase					
	codigo	edad	ingreso	estado_civil	fecmatricula
0	9876654	26.0	2500	Soltero	25/07/2022
1	7665498	NaN	2500	Soltero	21/07/2022
2	6598764	30.0	3500	Casado	28/07/2022
3	5498766	42.0	4800	Casado	26/07/2022
4	8976654	38.0	4100	Soltero	22/07/2022



Pandas: Series y Dataframe

Analizar datos con Pandas

- Método head: Permite obtener los primeros registros de un Dataframe

```
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula
0	9876654	26.0	2500	Soltero	25/07/2022
1	7665498	NaN	2500	Soltero	21/07/2022

- Método tail: Permite obtener los últimos registros de un Dataframe

```
df_clase.tail(3)
```

	codigo	edad	ingreso	estado_civil	fecmatricula
2	6598764	30.0	3500	Casado	28/07/2022
3	5498766	42.0	4800	Casado	26/07/2022
4	8976654	38.0	4100	Soltero	22/07/2022

Pandas: Series y Dataframe

Analizar datos con Pandas

- Método Describe: Genera un resumen estadístico de los datos numéricos

```
df_clase.describe()
```

	codigo	edad	ingreso
count	5.000000e+00	4.000000	5.000000
mean	7.723267e+06	34.000000	3480.000000
std	1.762406e+06	7.302967	1005.982107
min	5.498766e+06	26.000000	2500.000000
25%	6.598764e+06	29.000000	2500.000000
50%	7.665498e+06	34.000000	3500.000000
75%	8.976654e+06	39.000000	4100.000000
max	9.876654e+06	42.000000	4800.000000



Pandas: Series y Dataframe

Analizar datos con Pandas

- Método Info: Obtiene información de cada campo del Dataframe

```
df_clase.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   codigo          5 non-null     int64
1   edad            4 non-null     float64
2   ingreso         5 non-null     int64
3   estado_civil    5 non-null     object
4   fecmatricula    5 non-null     object
dtypes: float64(1), int64(2), object(2)
memory usage: 328.0+ bytes
```



Pandas: Series y Dataframe

Analizar datos con Pandas

- Método isnull - isna: Permite detectar valores faltantes, retorna booleanos (isnull es un alias de isna)

```
df_clase.isnull()
```

	codigo	edad	ingreso	estado_civil	fecmatricula
0	False	False	False	False	False
1	False	True	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False



Pandas: Series y Dataframe

Analizar datos con Pandas

- **Método Any:** Devuelve verdadero si algún elemento es verdadero. Es utilizado frecuentemente con el método isnull.

Seguido de isnull, en un Dataframe, devuelve qué columnas tienen algún valor faltante.

```
df_clase.isnull().any()

codigo      False
edad        True
ingreso      False
estado_civil False
fecmatricula False
dtype: bool
```

Seguido de isnull y isany, en un Dataframe, devuelve si el dataframe tiene algún valor faltante

```
df_clase.isnull().any().any()

True
```



Pandas: Series y Dataframe

Analizar datos con Pandas

- Método Unique: Retorna valores únicos de una serie.

```
df_clase['ingreso'].unique()
array([2500, 3500, 4800, 4100])
```

- Método Nunique: Cuenta el número de elementos distintos en un dataframe.

```
// Cuenta el número
df_clase.nunique()

codigo      5
edad        4
ingreso      4
estado_civil 2
fecmatricula 5
dtype: int64
```



Pandas: Series y Dataframe

Propiedades de un Dataframe

- Shape: Indica el tamaño del Dataframe (número de filas y columnas en formato dupla)

```
df_clase.shape
(2, 5)
```

- Columns: Muestra los valores de las columnas de un Dataframe

```
df_clase.columns
Index(['codigo', 'edad', 'ingreso', 'estado_civil', 'fecmatricula'], dtype='object')
```

- Index: Muestra la secuencia de los índices de un Dataframe.

```
df_clase.index
RangeIndex(start=0, stop=5, step=1)
```

Pandas: Series y Dataframe

Trabajando con Series

- Seleccionando series a partir de un Dataframe: Se obtiene seleccionando un campo sólo un par de corchetes.

```
type(df_clase['ingreso'])
pandas.core.series.Series

df_clase['ingreso']
0    2500
1    2500
2    3500
3    4800
4    4100
Name: ingreso, dtype: int64
```



Pandas: Series y Dataframe

Trabajando con Series

- Slicing: Permite extraer partes de una serie.

```
df_clase['ingreso'][1:3]
```

1	2500
2	3500

Name: ingreso, dtype: int64

```
df_clase['ingreso'][3:]
```

3	4800
4	4100

Name: ingreso, dtype: int64

```
df_clase.ingreso[:2]
```

0	2500
1	2500

Name: ingreso, dtype: int64



Pandas: Series y Dataframe

Seleccionando campos

- Subsetting Dataframes: Es el proceso de seleccionar un conjunto de filas y columnas de un dataframe dando como resultado un segundo Dataframe.

```
df_clase[['ingreso', 'codigo', 'edad']]
```

	ingreso	codigo	edad
0	2500	9876654	26.0
1	2500	7665498	NaN
2	3500	6598764	30.0
3	4800	5498766	42.0
4	4100	8976654	38.0

```
df_clase[['codigo', 'ingreso', 'edad']][:3]
```

	codigo	ingreso	edad
0	9876654	2500	26.0
1	7665498	2500	NaN
2	6598764	3500	30.0



Agrupaciones en Pandas

- Método groupby
- Agrupaciones directas
- Agrupaciones por agregación
- Agrupaciones independientes al campo
- Agrupación múltiple
- Bono: Renombrado de columnas
- Bono: Reseteo de índices



Agrupaciones en Pandas

Dataframe df_base

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False
5	5	Charmeleon	Fire	NaN	405	58	64	58	80	65	80	1	False
6	6	Charizard	Fire	Flying	534	78	84	78	109	85	100	1	False
7	6	CharizardMega Charizard X	Fire	Dragon	634	78	130	111	130	85	100	1	False
8	6	CharizardMega Charizard Y	Fire	Flying	634	78	104	78	159	115	100	1	False
9	7	Squirtle	Water	NaN	314	44	48	65	50	64	43	1	False
10	8	Wartortle	Water	NaN	405	59	63	80	65	80	58	1	False
11	9	Blastoise	Water	NaN	530	79	83	100	85	105	78	1	False
12	9	BlastoiseMega Blastoise	Water	NaN	630	79	103	120	135	115	78	1	False
13	10	Caterpie	Bug	NaN	195	45	30	35	20	20	45	1	False
14	11	Metapod	Bug	NaN	205	50	20	55	25	25	30	1	False
15	12	Butterfree	Bug	Flying	395	60	45	50	90	80	70	1	False



Agrupaciones en Pandas

- **Método groupby:**

Permite agrupar grandes cantidades de datos y calcular operaciones en estos grupos.

```
gb_base = df_base.groupby('Type 1')
type(gb_base)

pandas.core.groupby.generic.DataFrameGroupBy
```



Agrupaciones en Pandas

- **Agrupaciones directas:** Permite mostrar el resultado de aplicar una función a un Dataframe. Por defecto afectará a todas sus columnas.

Nomenclatura:

`<df>.groupby(<Col. a agrupar>)[[<Col. 1>, <Col. 2>, ...]].<función>`

```
df_base.groupby('Type 1')[['Attack', 'Sp. Atk']].max()
```

	Attack	Sp. Atk
Type 1		
Bug	45	90
Fire	130	159
Grass	100	122
Water	103	135



Agrupaciones en Pandas

- **Agrupaciones por agregación:** Permite aplicar multiples funciones al campo agrupado utilizando el método agg.

```
gb_base = df_base.groupby('Type 1')[['Attack', 'Sp. Atk']]
gb_base.agg(['count', 'min', 'max'])
```



	Attack			Sp. Atk		
	count	min	max	count	min	max
Type 1						
Bug	3	20	45	3	20	90
Fire	5	52	130	5	60	159
Grass	4	49	100	4	65	122
Water	4	48	103	4	50	135



Agrupaciones en Pandas

- **Agrupaciones independientes al campo:**
Permite aplicar diferentes funciones a cada campo.

```
gb_base = df_base.groupby('Type 1')[['#', 'Attack', 'Sp. Atk']]
gb_base.agg({'#': 'count', 'Attack': ['min', 'max'], 'Sp. Atk': 'max'})
```

	#	Attack		Sp. Atk
	count	min	max	max
Type 1				
Bug	3	20	45	90
Fire	5	52	130	159
Grass	4	49	100	122
Water	4	48	103	135



Agrupaciones en Pandas

- **Agrupación múltiple:**

Permite realizar varias agrupaciones a un DataFrame.

```
gb_base = df_base.groupby(['Type 1', 'Type 2'])[['Attack', 'Sp. Atk']]
gb_base = gb_base.agg({'Attack': 'max', 'Sp. Atk': ['min', 'max']})
gb_base
```

		Attack	Sp. Atk	
		max	min	max
Type 1	Type 2			
Bug	Flying	45	90	90
Fire	Dragon	130	130	130
	Flying	104	109	159
Grass	Poison	100	65	122



Agrupaciones en Pandas

Bono: Renombrado de columnas

- Renombrado de **TODAS** las columnas: Se utiliza la propiedad `columns` y se le asigna una lista con los valores de las columnas a renombrar.

```
gb_base.columns = ['max_attack', 'min_sp_attack', 'max_sp_attack']
gb_base
```

		max_attack	min_sp_attack	max_sp_attack	
Type 1	Type 2				
Bug	Flying	45	90	90	
Fire	Dragon	130	130	130	
	Flying	104	109	159	
Grass	Poison	100	65	122	



Agrupaciones en Pandas

Bono: Renombrado de columnas

- Renombrado de **DETERMINADAS** las columnas: Se utiliza el método `rename` y se asignan las columnas a renombrar mediante un diccionario.

```
gb_base.rename(columns={'min_sp_attack': 'min_spatk', 'max_sp_attack': 'max_spatk'}, inplace=True)
gb_base
```

		max_attack	min_spatk	max_spatk
Type 1	Type 2			
Bug	Flying	45	90	90
Fire	Dragon	130	130	130
	Flying	104	109	159
Grass	Poison	100	65	122

Agrupaciones en Pandas

Bono: Reseteo de índices

Se utiliza el método `reset_index`, y permitirá reiniciar la secuencia numérica de los índices. Los antiguos índices son insertados como una nueva columna del DataFrame.

```
gb_base.reset_index(inplace=True)
gb_base
```

	Type 1	Type 2	max_attack	min_spatk	max_spatk
0	Bug	Flying	45	90	90
1	Fire	Dragon	130	130	130
2	Fire	Flying	104	109	159
3	Grass	Poison	100	65	122



Caso de Uso: RetailMax

RetailMax guarda datos históricos de ventas en archivos que provienen de distintos sistemas. Estos datos se usarán para un análisis de desempeño mensual, pero presentan varios problemas:

- Algunas columnas tienen tipos de datos incorrectos (por ejemplo, precios como texto).
- Existen valores faltantes en campos importantes como cantidad o fecha de venta.
- Se incluyen columnas irrelevantes que no aportan al análisis (ej. código interno del sistema).

El problema es que, en su estado actual, los datos no pueden usarse directamente para análisis, lo que genera errores y resultados poco confiables.

Caso de Uso: RetailMax

Con Python y pandas se puede:

- Cargar los datos históricos en un DataFrame.
- Convertir tipos de datos al formato correcto.
- Manejar valores faltantes.
- Omitir las columnas irrelevantes.

Trabajo



Trabajo: Agrupaciones en Pandas

- Obtener el máximo y mínimo ataque físico (Attack) de cada tipo de Pokemon (Type 1). Utilizar el siguiente dataset: <https://raw.githubusercontent.com/AngelTintaya/datasets/main/pokemon.csv>
- El objetivo es obtener un DataFrame como el siguiente:

	Type 1	min_attack	max_attack
0	Bug	10	185
1	Dark	50	150
2	Dragon	50	180
3	Electric	30	123
4	Fairy	20	131



¡GRACIAS!

